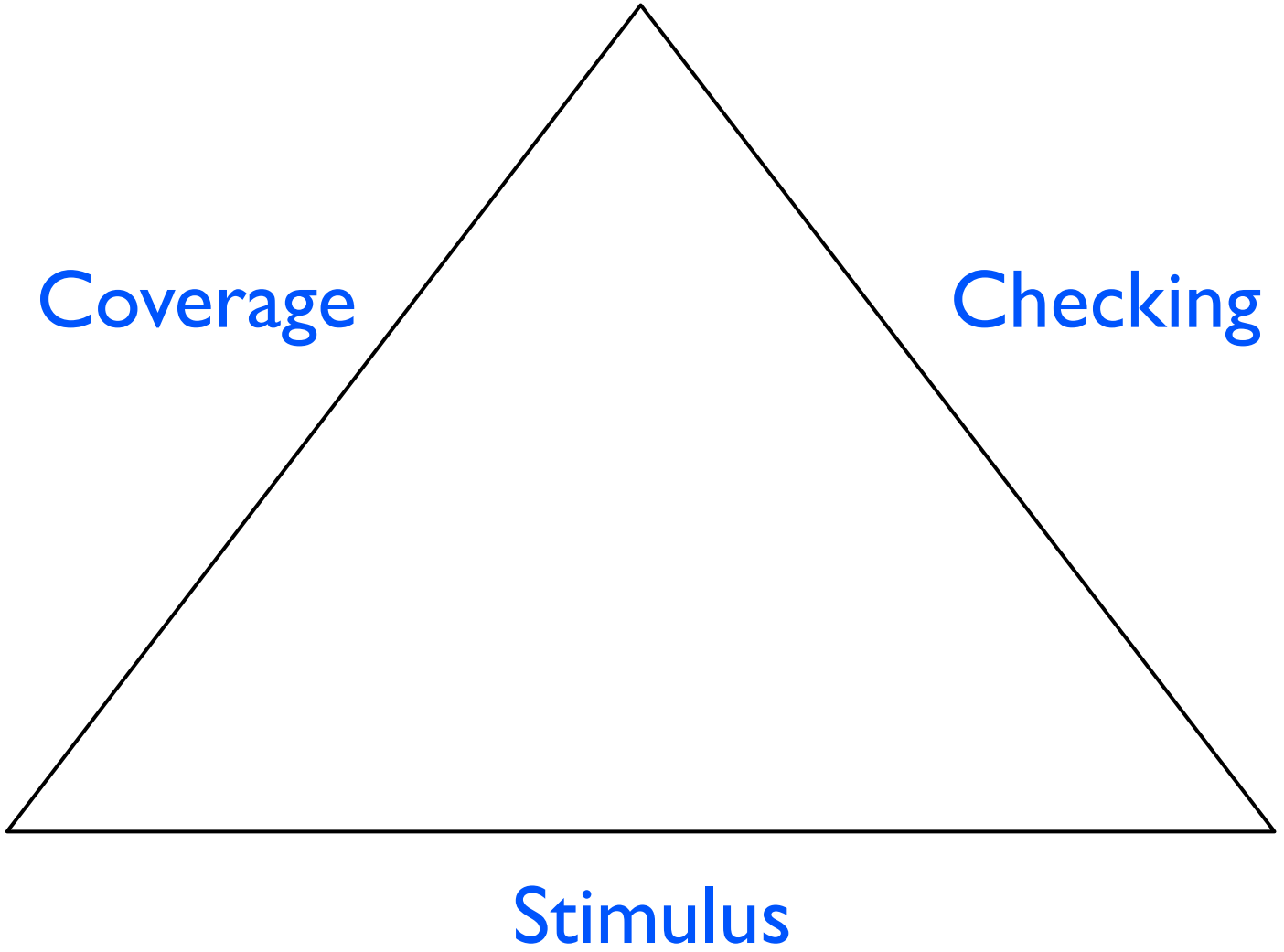
CPU Verification

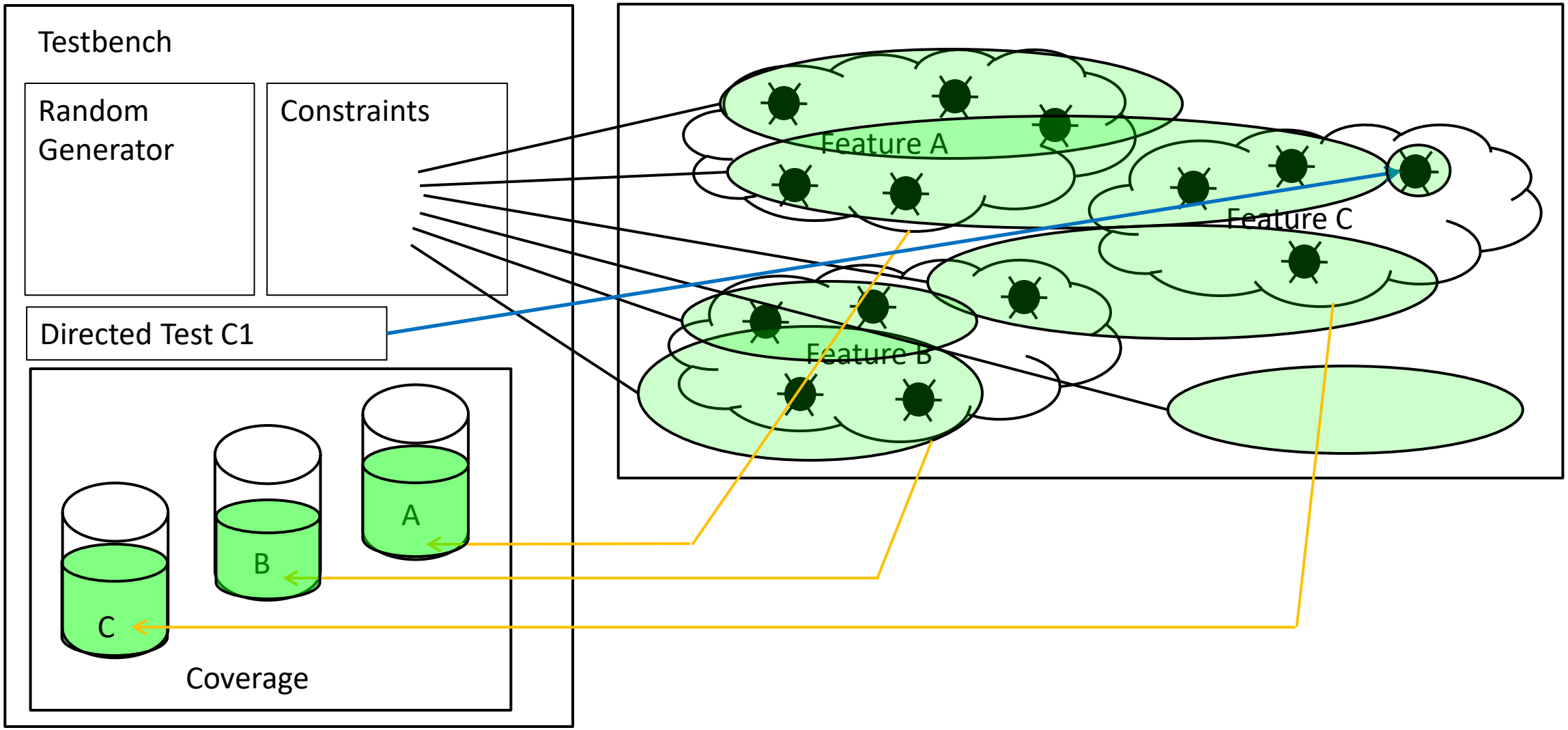
(Many slide are taken from Hitesh Mishra's slides on processor verification at ARM)

General Concepts

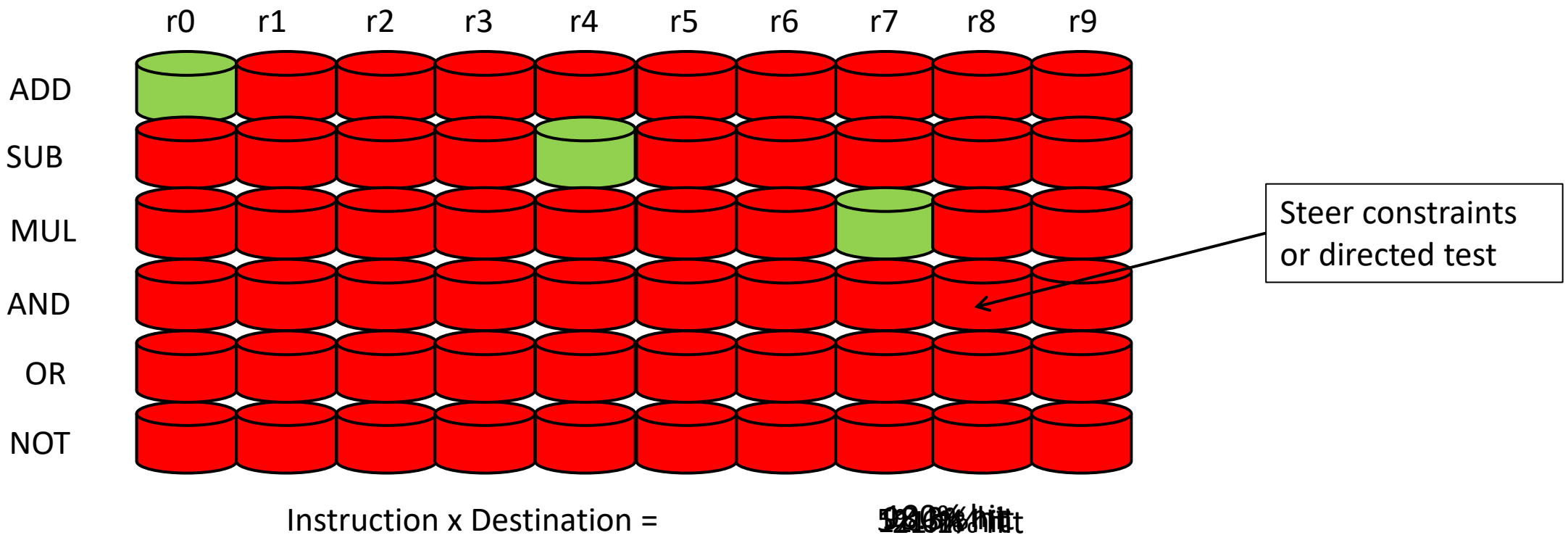
The Key to Successful Verification



Coverage-Driven Constrained Random Verification + Directed Tests



Functional Coverage



Coverage: Are we done?

“Once we get to 100% Code Coverage and 100% Functional Coverage are we done?”

- No! Functional Coverage is only as good as the coverage points you define. There may be bugs in scenarios or relationships between features that have not been thought of and have not yet been stimulated by the constrained random stimulus.

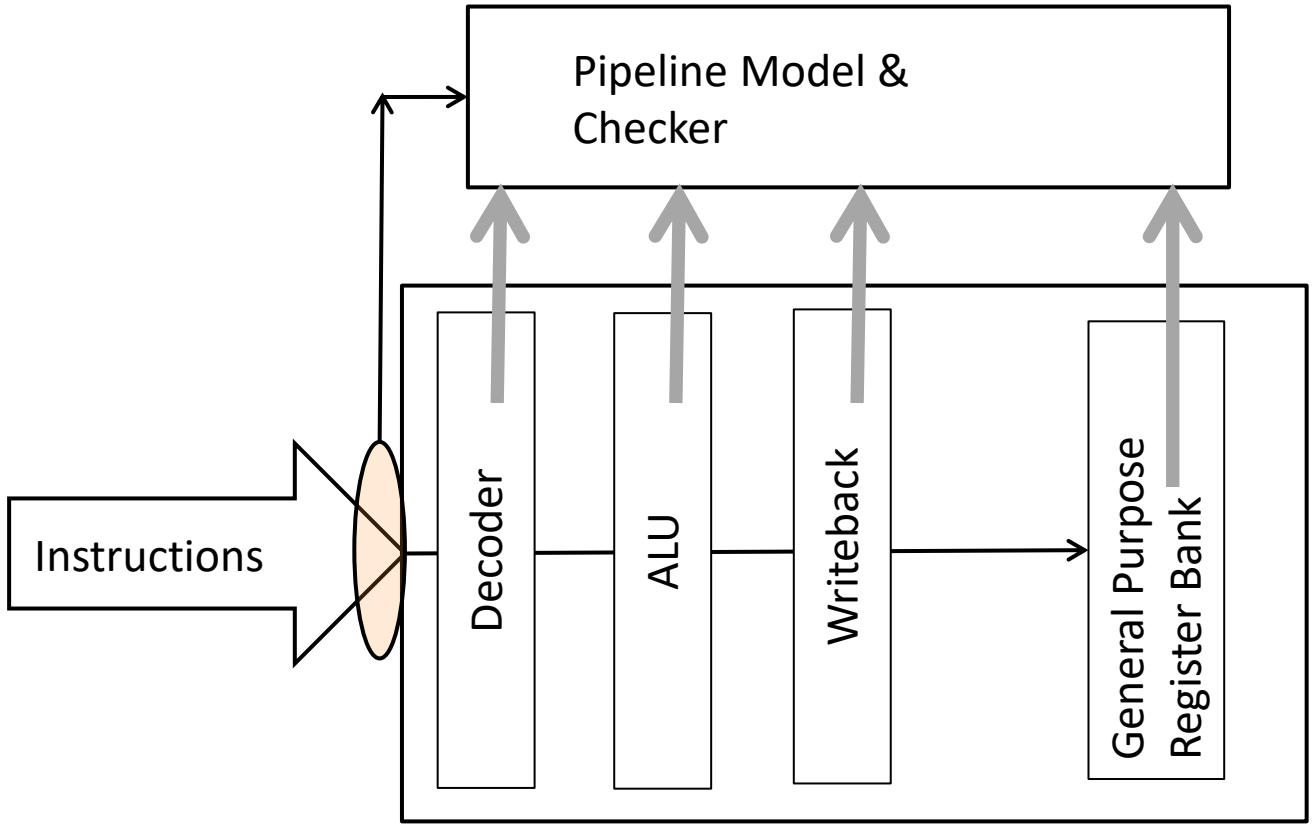
“How do we solve this?”

- We can never be fully done… BUT… we aim to be.
- To achieve this we run extended soak verification and overlapping verification techniques... Essentially lots more cycles to ensure that no new bugs pop out in unexpected areas.

“And if we get new bugs?”

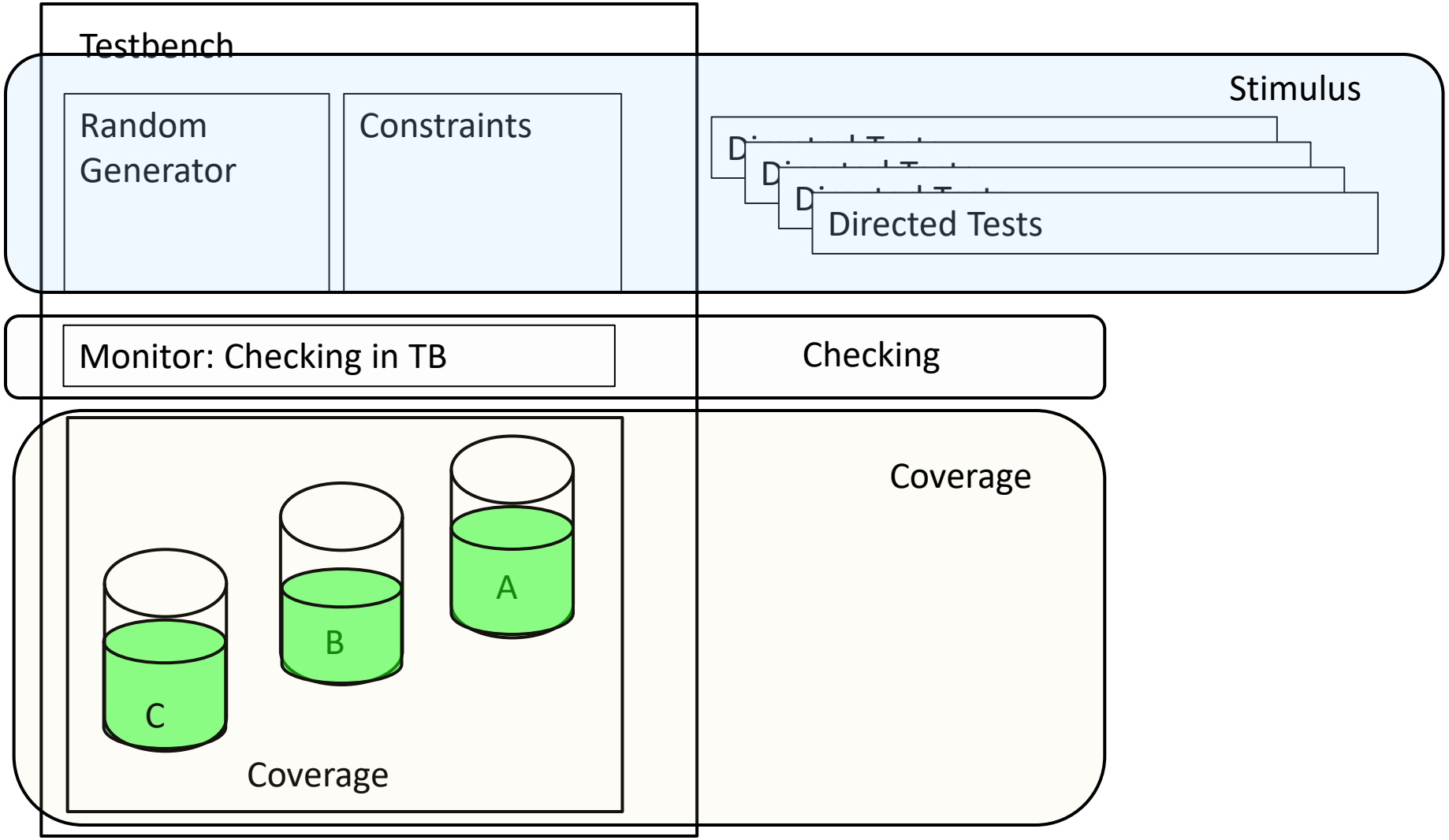
- Augment the functional coverage such that we know these scenarios are hit in future runs.

Checking



- The Grey arrow indicates “grey box” checking, whereby some internal state needs to be probed to perform the check
- Result Modelling: In this case, we grab the inputs and need to perform some transformation/ algorithm to obtain the value to check against the grey values
- Accurate modelling is required to compute the result & check DUT is correct

Complete Testbench



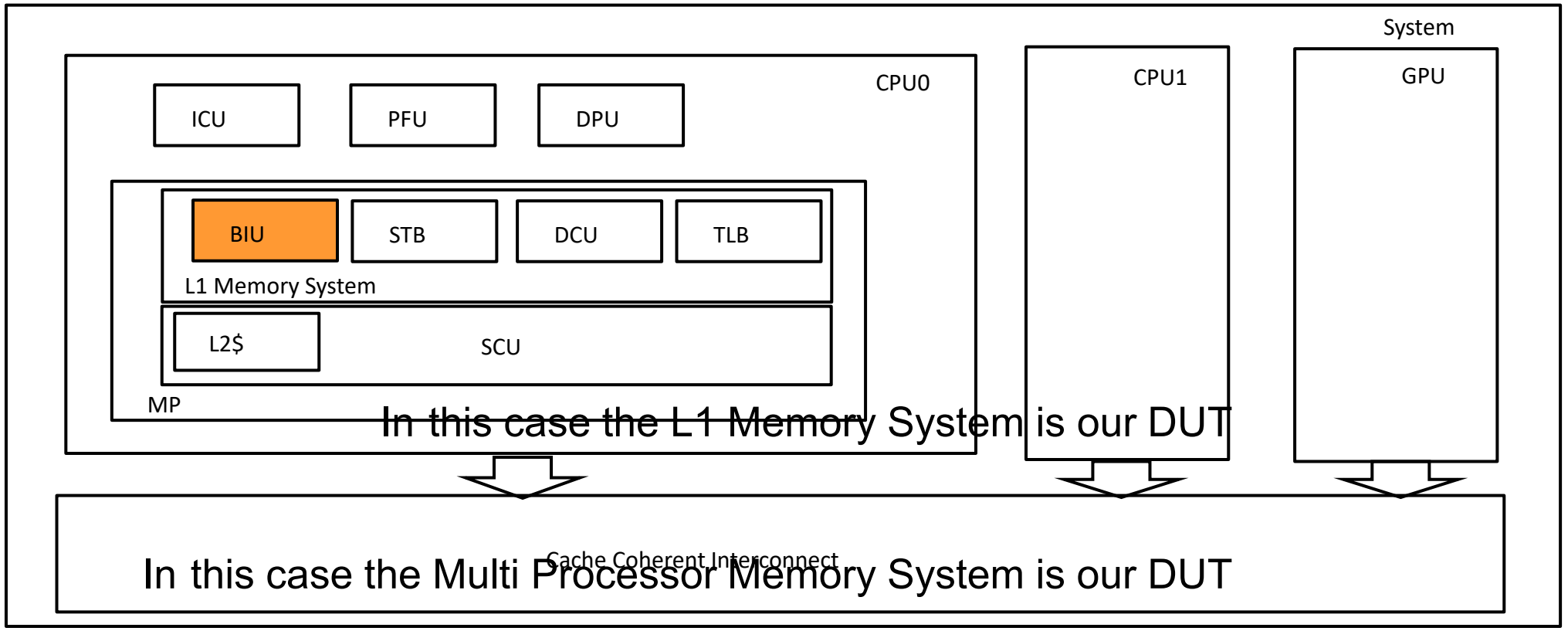
Multi-Layered Verification

9

Multi-Layered Verification Strategy

System verification	• Interoperability • Baremetal stress verification • OS boot + application stress
Top-level verification	• Architectural Suits • Directed Tests • Random Tests
Multi-unit	• Multi-unit level testbenches for major subsystems
Unit	• Unit level testbenches • Formal proofs
Sub-unit	• Sub-unit testbenches • Formal proofs

Bottom-Up Verification Methodology

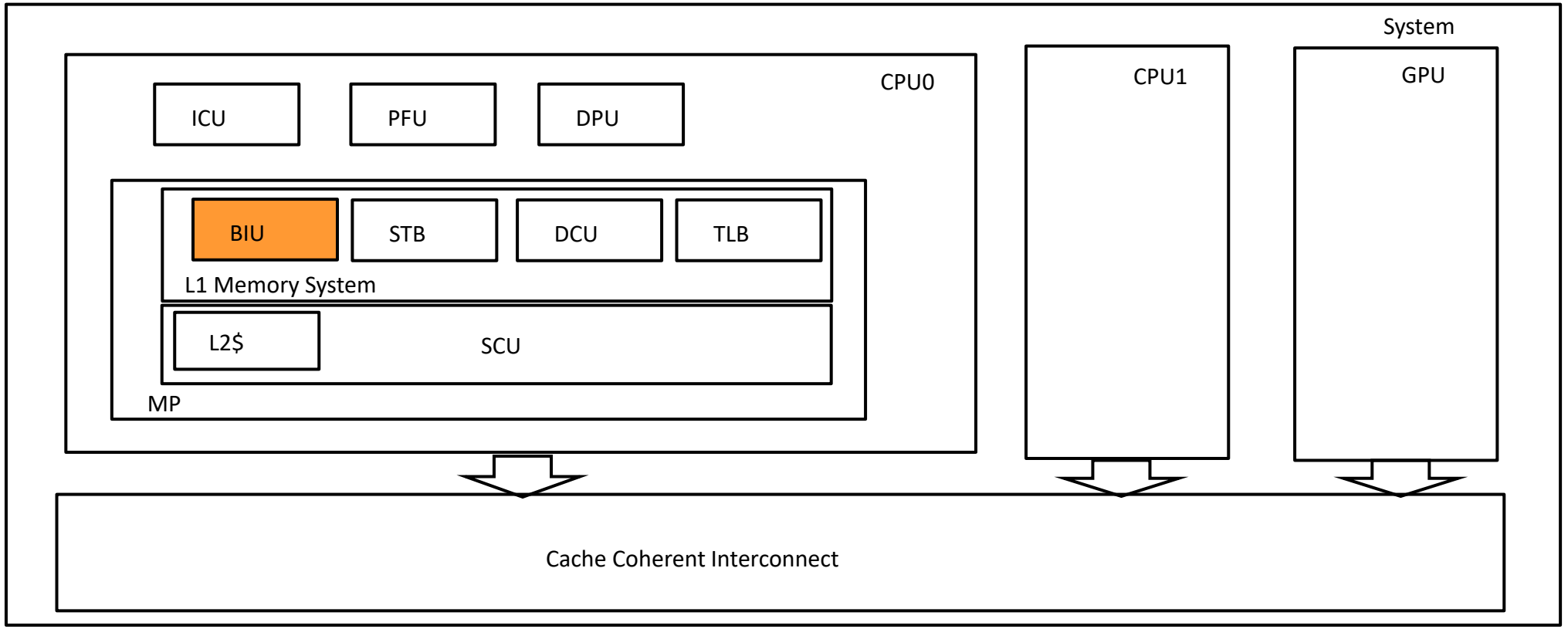


In this case the Bus Interface Unit (BIU) is our **D**evice **U**nder **T**est (DUT)

In this case the full system is the DUT

In this case CPU0 is the DUT

Bottom-Up Verification Methodology

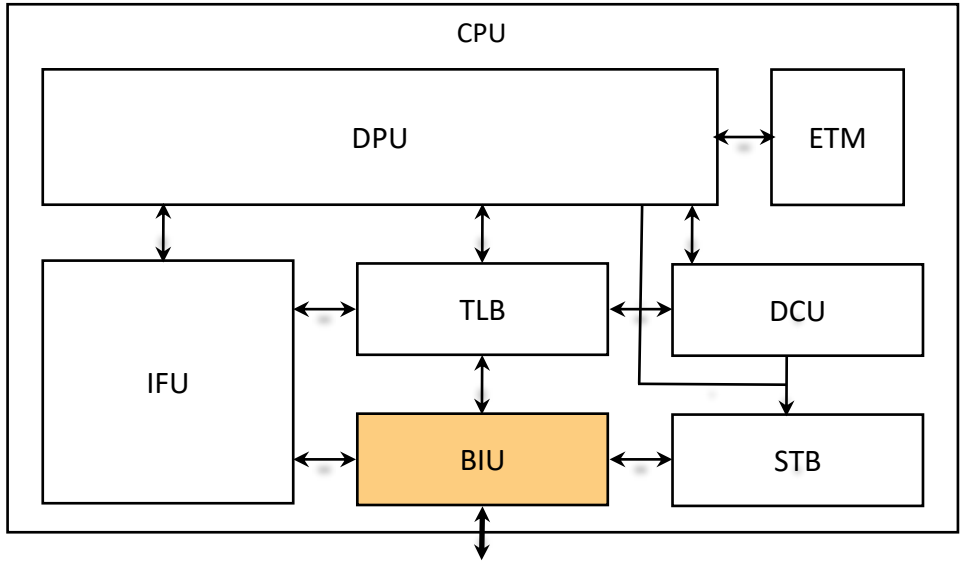


- Exhaust finding bugs at the lowest level before moving up a level of abstraction
- BIU tested in Unit Level TB, L1 Multi-Unit TB, MP Multi-Unit TB and Top level TB
- Also tested as part of a wider System Level TB including other CPUs, GPUs, Memory Controllers and Coherent Interconnects

Unit Level Verification

Exhaust finding bugs at the lowest level before moving up a level of abstraction

- Using the CPU's Bus Interface Unit (BIU) as an example, the following slides illustrate the levels of verification carried out
- The BIU is tested in a standalone testbench with Bus Functional Models (BFMs) to mimic the interactions with the rest of the DUT
  - This provides the quickest path to initial bring-up and basic operation bugs
- Once this technique saturates, we move to multi-unit verification

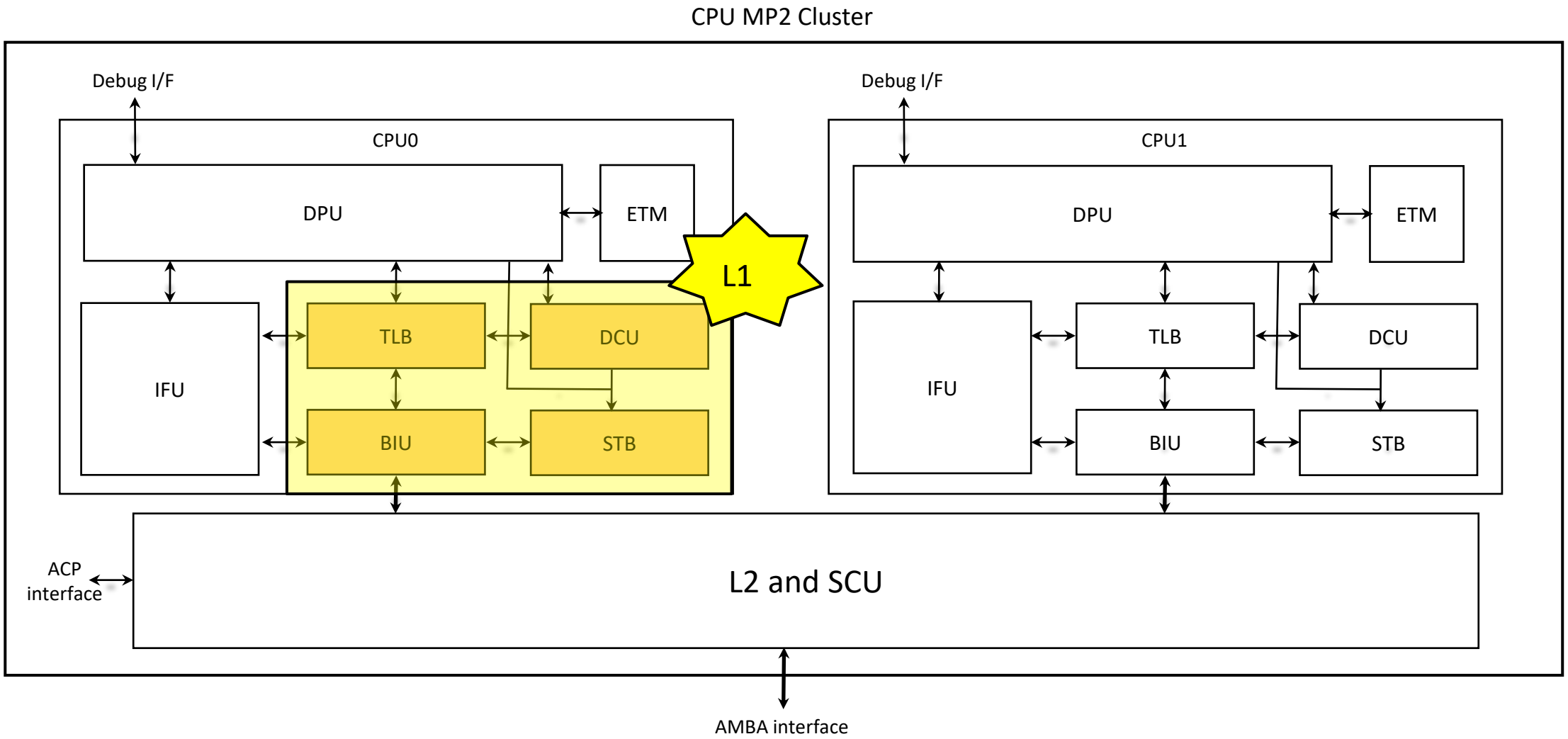


CPU Multi-Unit Verification

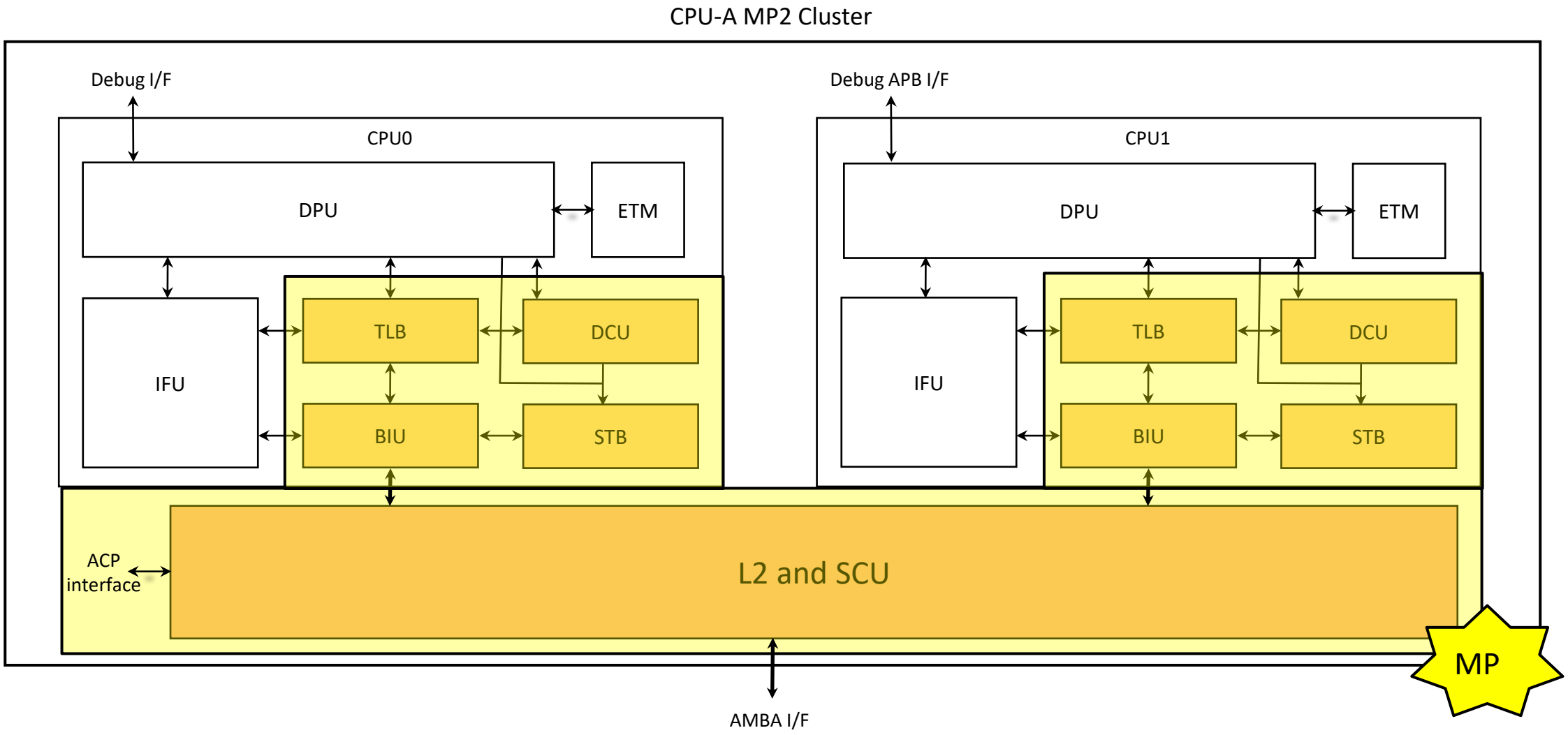
Verification of the complex interactions within a coherent memory system

- Both our Unit and Multi-Unit testbenches use constrained random stimulus
  - Fine-grained control of stimulus at different levels of hierarchy
  - Improves coverage of low-level state-space
  - Improves coverage of unit interactions
- In the CPU example there are two Multi-Unit testbenches as follows:
- L1 Testbench
  - Tests single-core data side memory system in isolation
  - Includes checks from DCU, STB, TLB, BIU unit-level testbenches
  - Additional architectural checks
- MP Testbench
  - Tests coherency between multiple data side memory systems
  - Includes checks from DCU, STB, TLB, BIU, SCU, L1 testbenches
  - Additional architectural checks
  - Wide range of configurations (multi-core, multi-cluster)
    - MP1, no L2 cache (simplest configuration)
    - MP4 + MP4 + CCI, with GIC and L2 caches (largest AMBA 4 ACE-based configuration)
    - MP4 + MP4 + MP4 + MP4 + CCN, with GIC and L2 caches (Largest AMBA 5 CHI-based configuration)

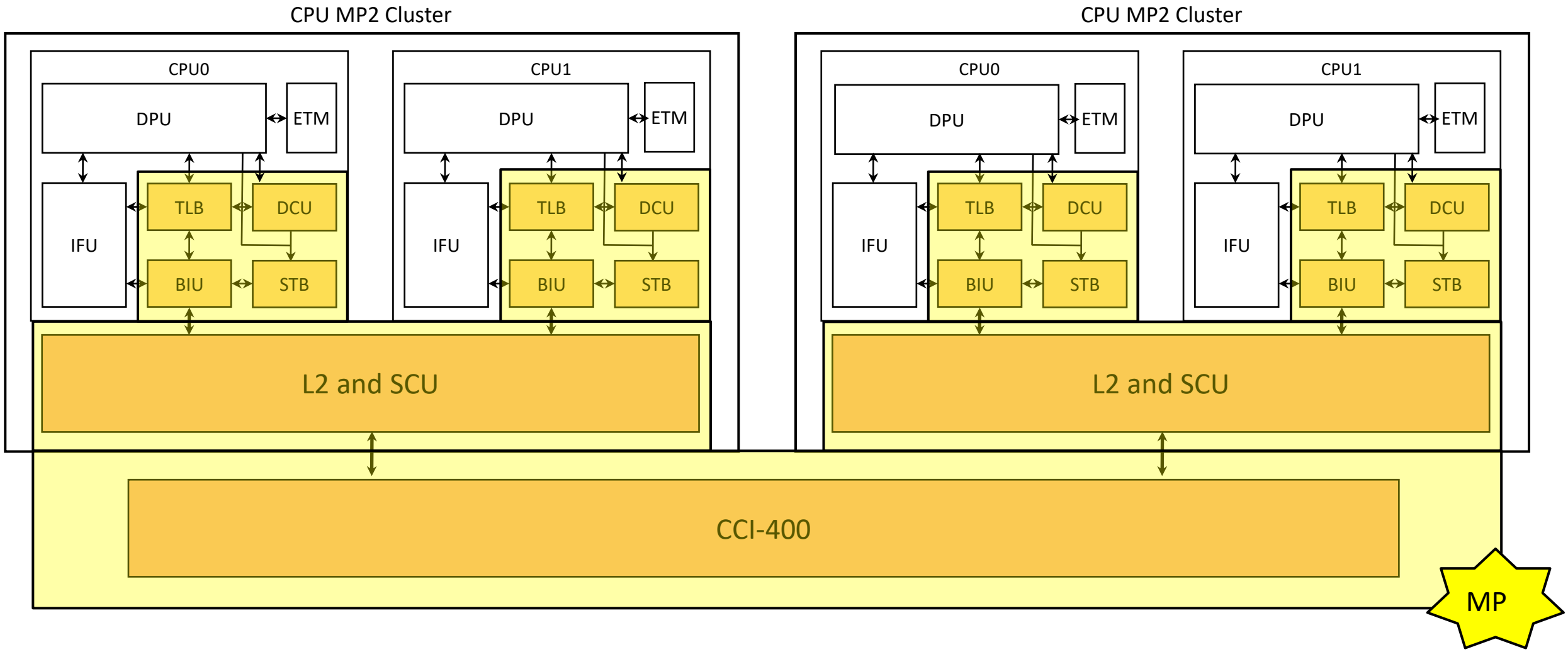
CPU Multi-Unit Verification



CPU Multi-Unit Verification Example



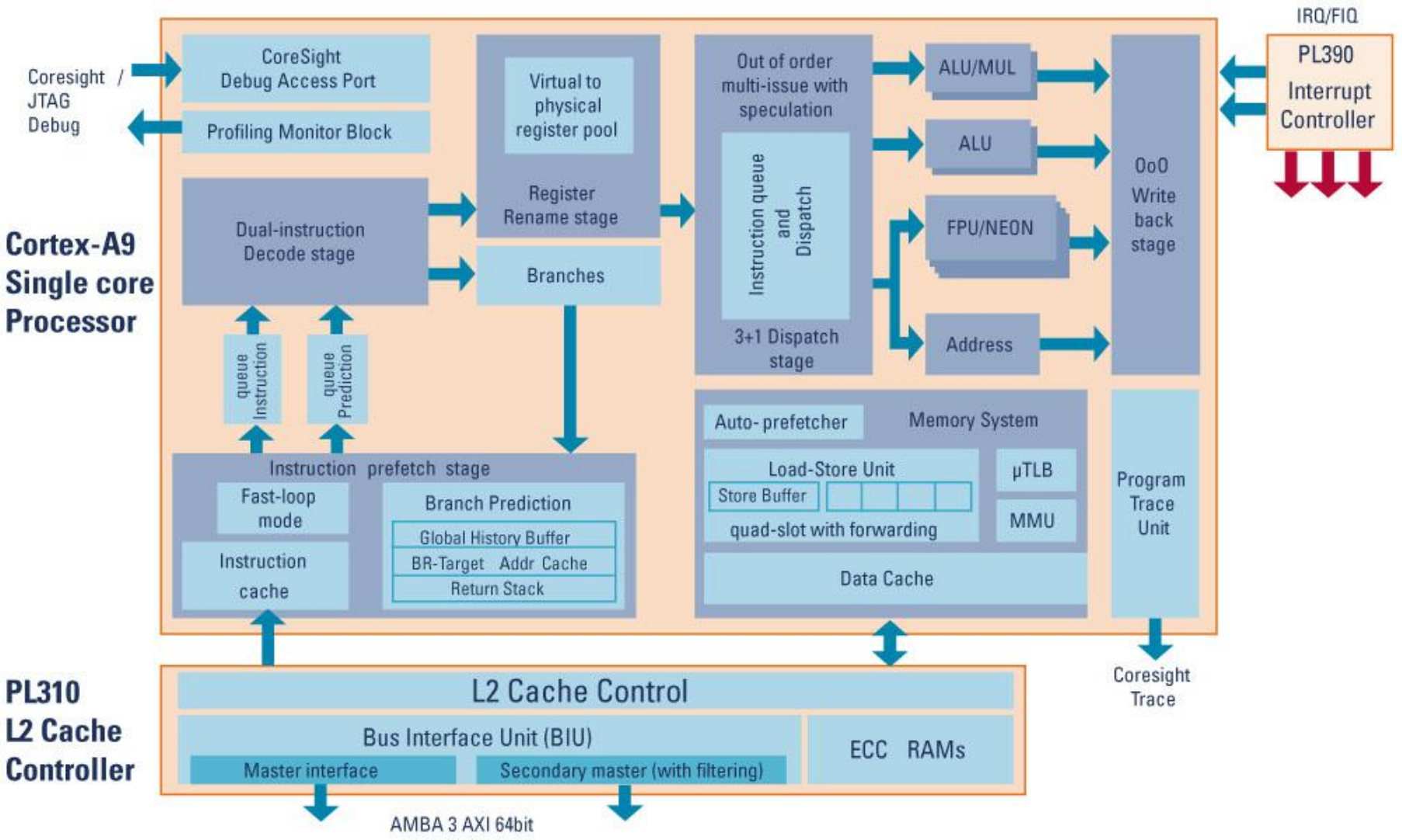
CPU Multi-Unit Verification Example



CPU Top-Level

- Scope typically includes
  - The core (all CPU pipeline stages)
  - Memory subsystem
  - Interfaces (interrupt controller, power management block, etc)
- Methodology
  - Verify the complete design by executing programs while controlling its external environment.
- Testbench components
  - Program loader
  - Memory models
  - Bus agents
  - Coverage collectors
  - Protocol checkers
  - Arch state checkers

CPU Top-Level – Single Core



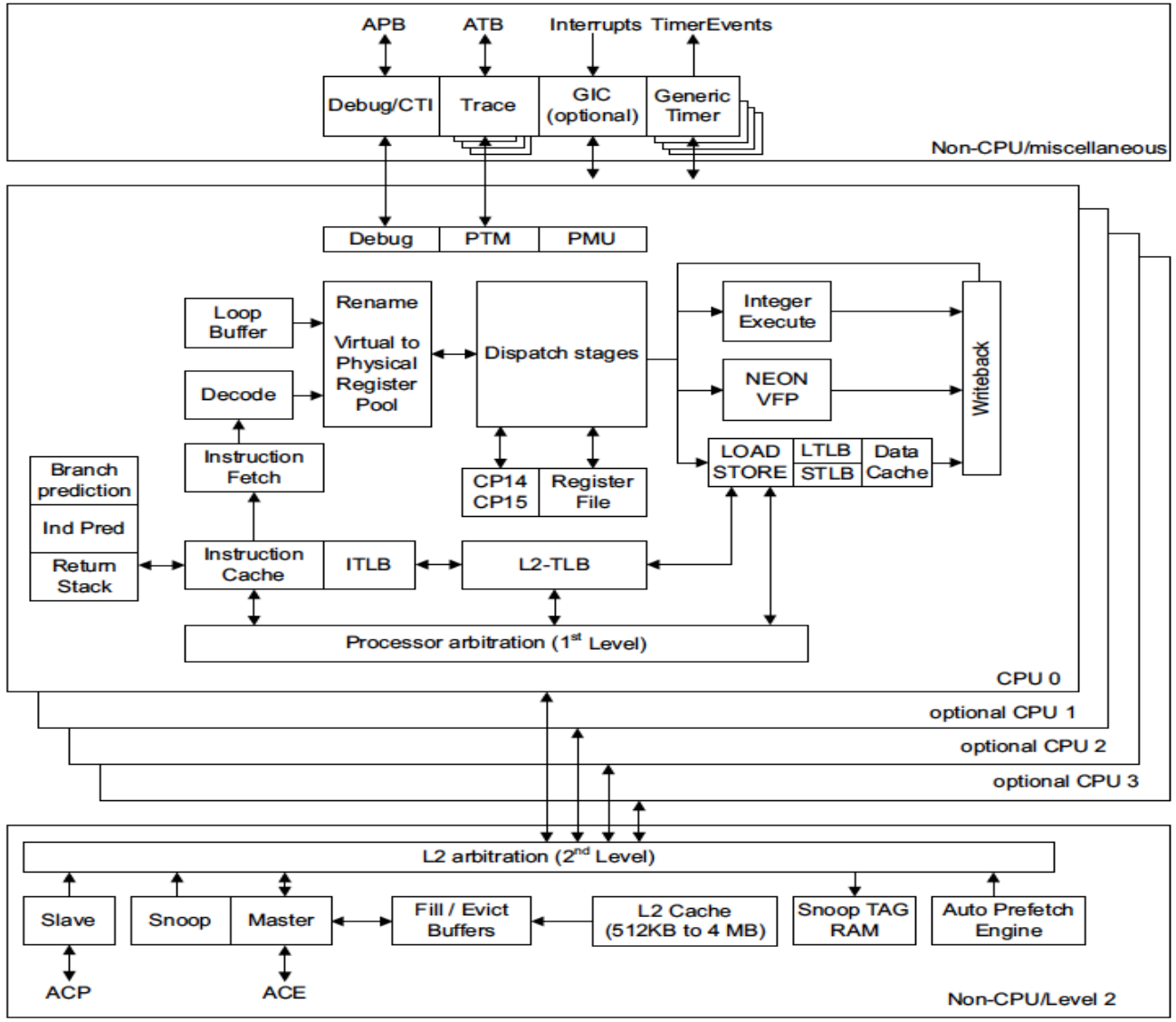
ARM Cortex-A15 MPCore Block Diagram

CPU Top-Level - Multicore

- **Global Interrupt Control, Trace and Debug handled across all cores**

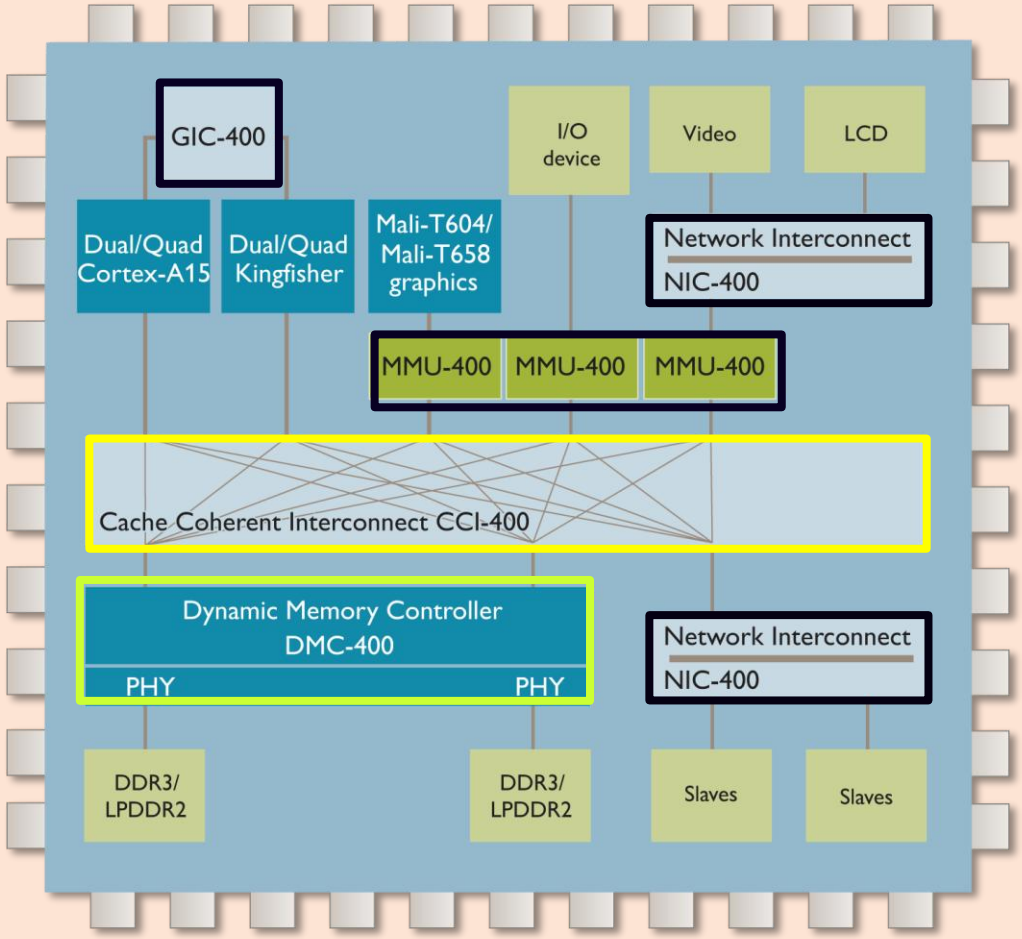
- **1-4 Processors per cluster**
- **Each processor has full Out-of-Order (OoO) pipeline.**

- **Integrated Level 2 cache**



ARM Cortex-A15 MPCore Block Diagram

System Level



Coherency

- CCI-400
  - Full cache coherency
  - I/O coherency
  - Prioritization and utilization

Virtualization

- MMU-400
  - OS level virtualization
- GIC-400
  - Virtual interrupts
  - Multicore support

External Memory Subsystem

- DMC-400
  - DDR utilization
  - PHY integration

Interconnect

- NIC-400
  - Routing efficiency

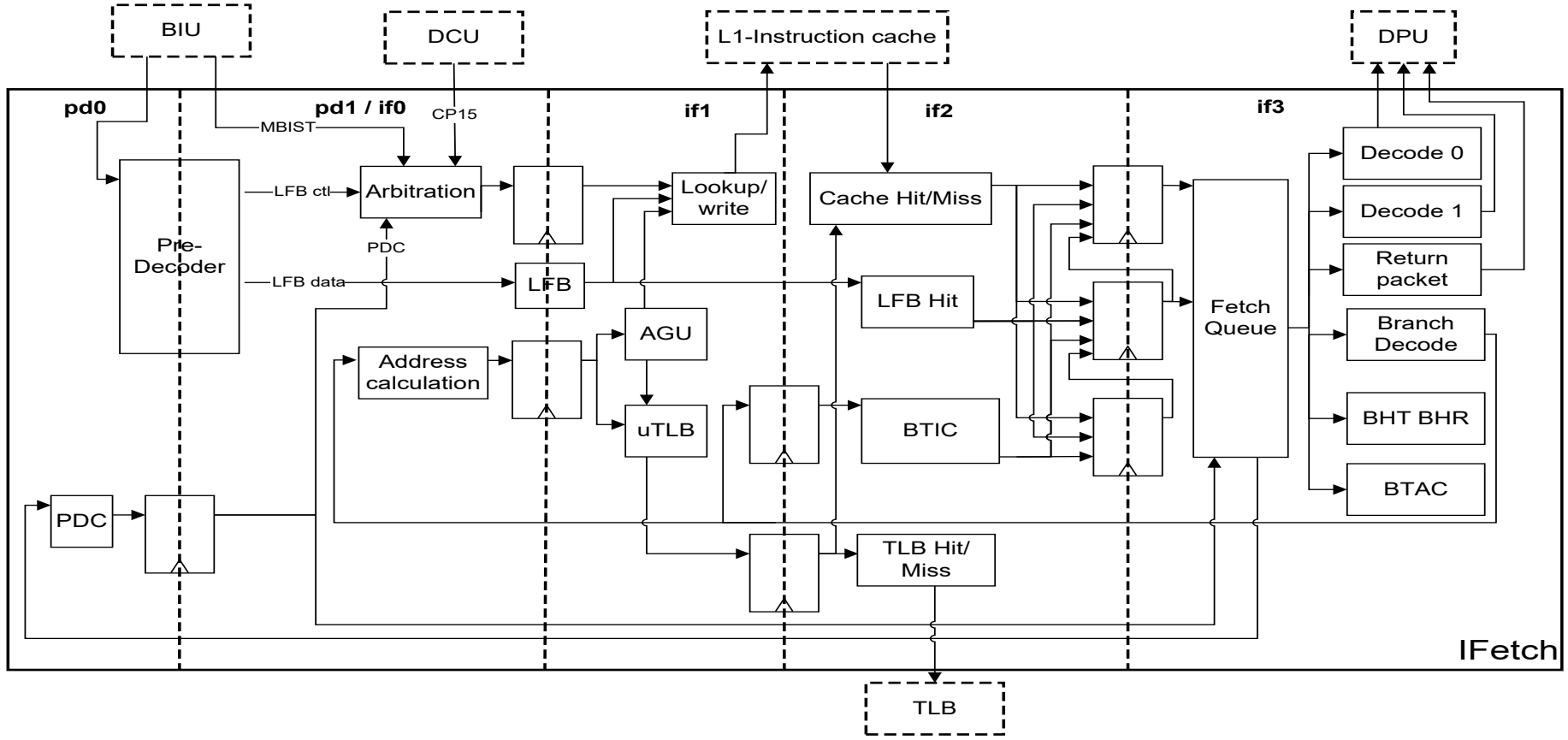
System Level: Operating Systems

- Aim: Run OS images and stress tests before release
  - … in short, make sure it all Just Works when a customer gets silicon and the current OS release
- Why?
  - to find bugs as early as possible – pain increases the later they're found
    - Sales sell the dream; implementation team provide the reality; we try to prevent it turning into a nightmare…
  - it gives early warning of updates needed in the 'big 3' OSes, either for bugs or new feature use
    - … and so lets us have patches ready along with the CPU release
  - it's what customers will do with real silicon, and it exposes problems not caught by other methods
    - … and makes sure features are actually usable in SW
- What:
  - Standard OS: Linux, Windows?
  - Graphics: driver stack, plus game traces/benchmarks/conformance suites
  - (plus some non-OS bare-metal memory and MP stress tests)

Unit-level Detailed Example

23

Instruction Fetch Unit (IFU) Example



Instruction Fetch Unit (IFU) Example

- Idealised Example:
- Data Processing Unit (DPU) sends an address to the IFU
- The IFU must then check whether the instruction at the requested address is in the Instruction Cache or requires to be fetched from main memory
  - If the instruction is fetched from main memory, it is then placed in the instruction cache for future use
  - A full cache line of instructions (512/32 = 16 instructions are fetched at a time)
- When the instruction is obtained externally or from the cache it must then be partially decoded to check if it is a branch
  - If so, branch prediction is performed and the output instruction to the DPU is modified with the result of the prediction
- Instructions are then placed in fetch queue, to be processed by the IFU for delivery to the DPU whenever the DPU makes a request for instructions

IFU Example

- To verify this IFU we must:
  - Stimulate the DUT with instructions
  - Stimulate the DUT with instructions
  - Model the Instruction Cache
  - Model the Branch Prediction algorithm
  - Check the instructions are fetched from the correct location
  - Check
  - Check
  - Measure
  - Measure that all instructions have been observed
    - Define

STIMULUS:

Need stimulus generator and BFMs

CHECKING

Need to model the Instruction Cache

Need to model the branch predictor

Need to monitor and the interfaces

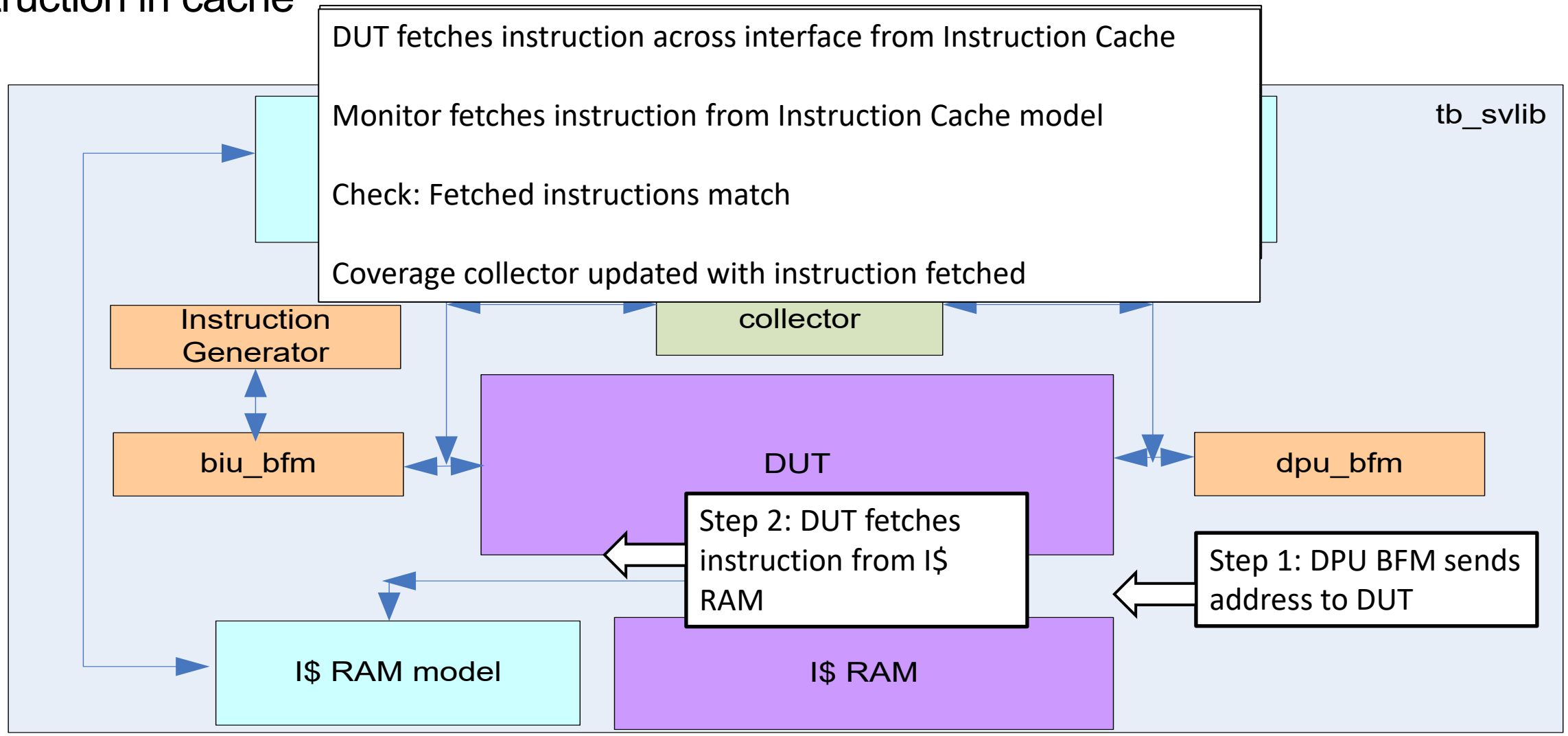
Need to add checks against expected behaviour

COVERAGE

Need to define the interesting scenarios

Need to probe at appropriate points of interest to get context of what has happened

IFU Example:
instruction in cache

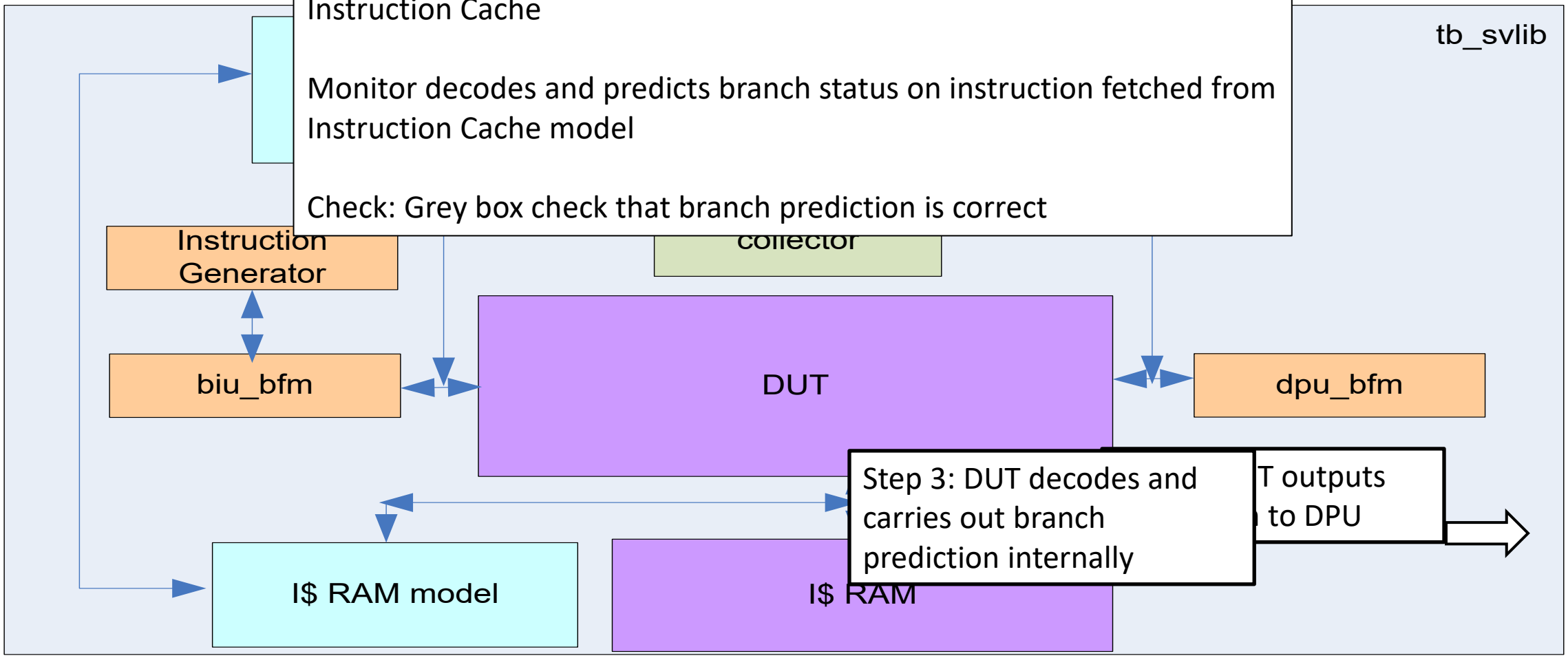


IFU Example:
instruction in cache

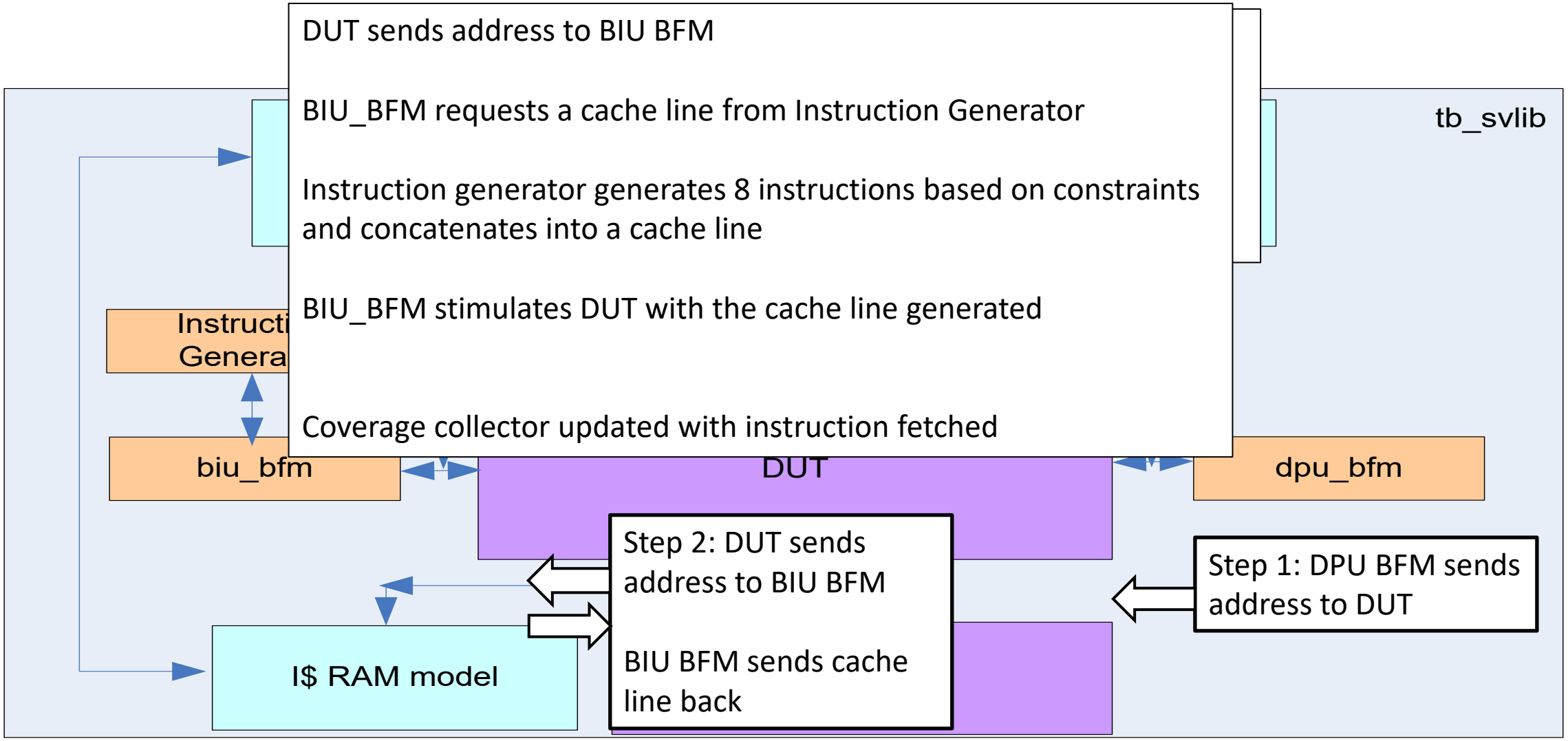
DUT decodes and predicts branch status on instruction fetched from Instruction Cache

Monitor decodes and predicts branch status on instruction fetched from Instruction Cache model

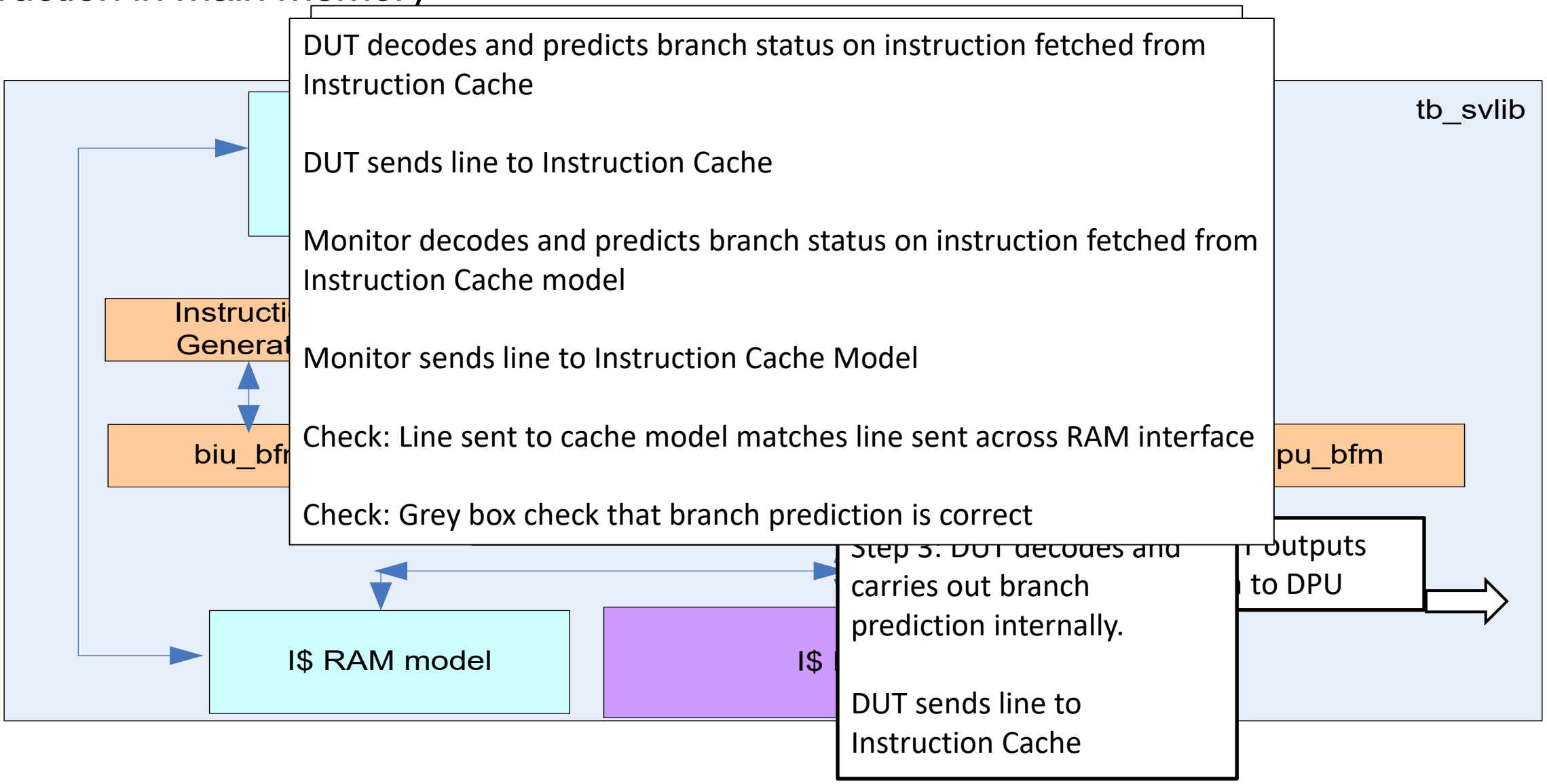
Check: Grey box check that branch prediction is correct



IFU Example:
instruction in main memory



IFU Example:
instruction in main memory



DUT decodes and predicts branch status on instruction fetched from Instruction Cache

DUT sends line to Instruction Cache

Monitor decodes and predicts branch status on instruction fetched from Instruction Cache model

Monitor sends line to Instruction Cache Model

Check: Line sent to cache model matches line sent across RAM interface

Check: Grey box check that branch prediction is correct

Step 3: DUT decodes and carries out branch prediction internally.

DUT sends line to Instruction Cache

IFU Example

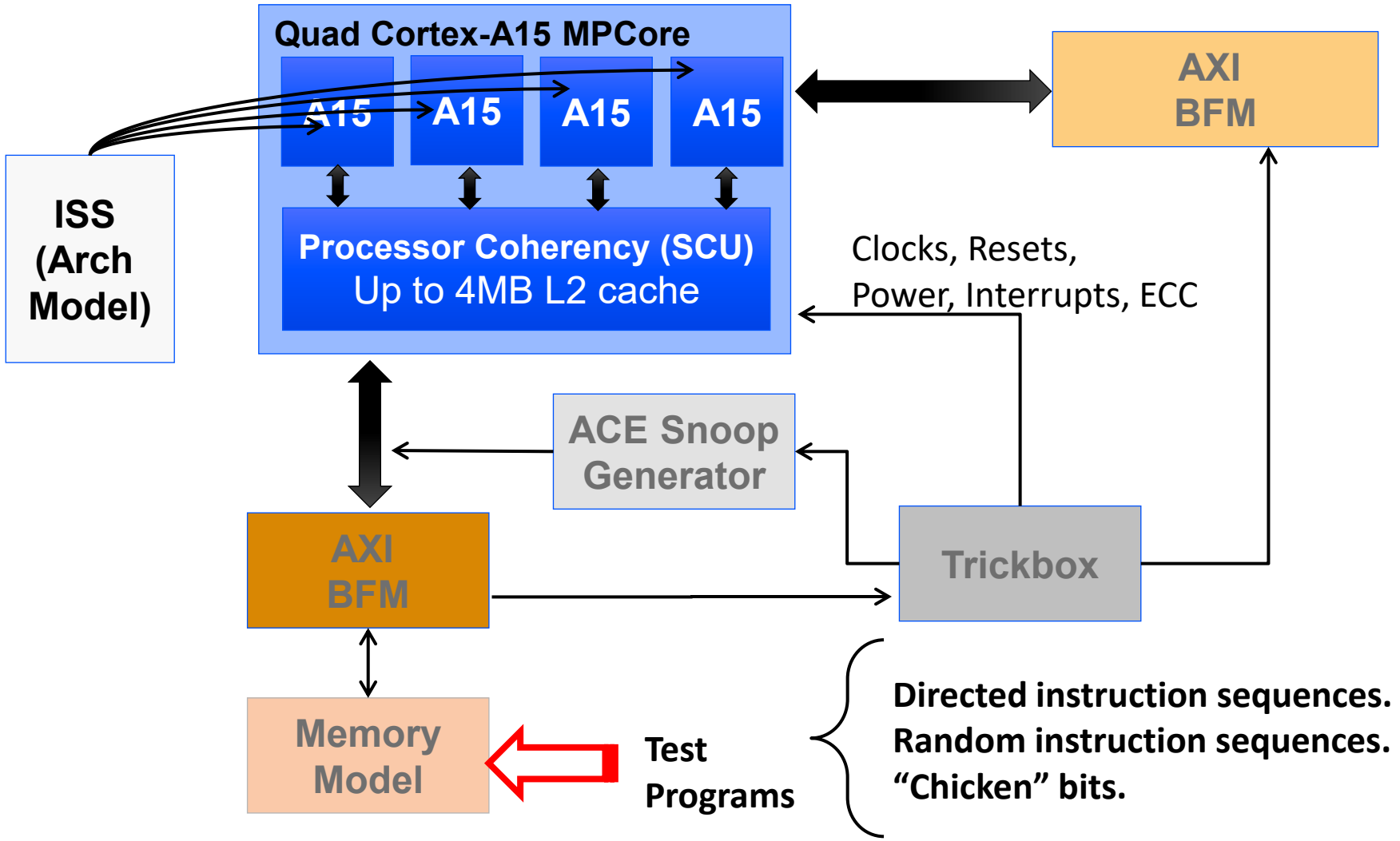
- Stimulus all generated in constrained random testbench
- Checking coming from both the testbench itself and from embedded assertions within the DUT
- Coverage collected by Coverage Collector for analysis offline in coverage tool
- Through random regression over a large number of seeds we will (hopefully) cover what we expect and...
- Uncover what we didn’t expect!

Top-level Detailed Example

32

CPU Top Level Testbench

Example from ARM

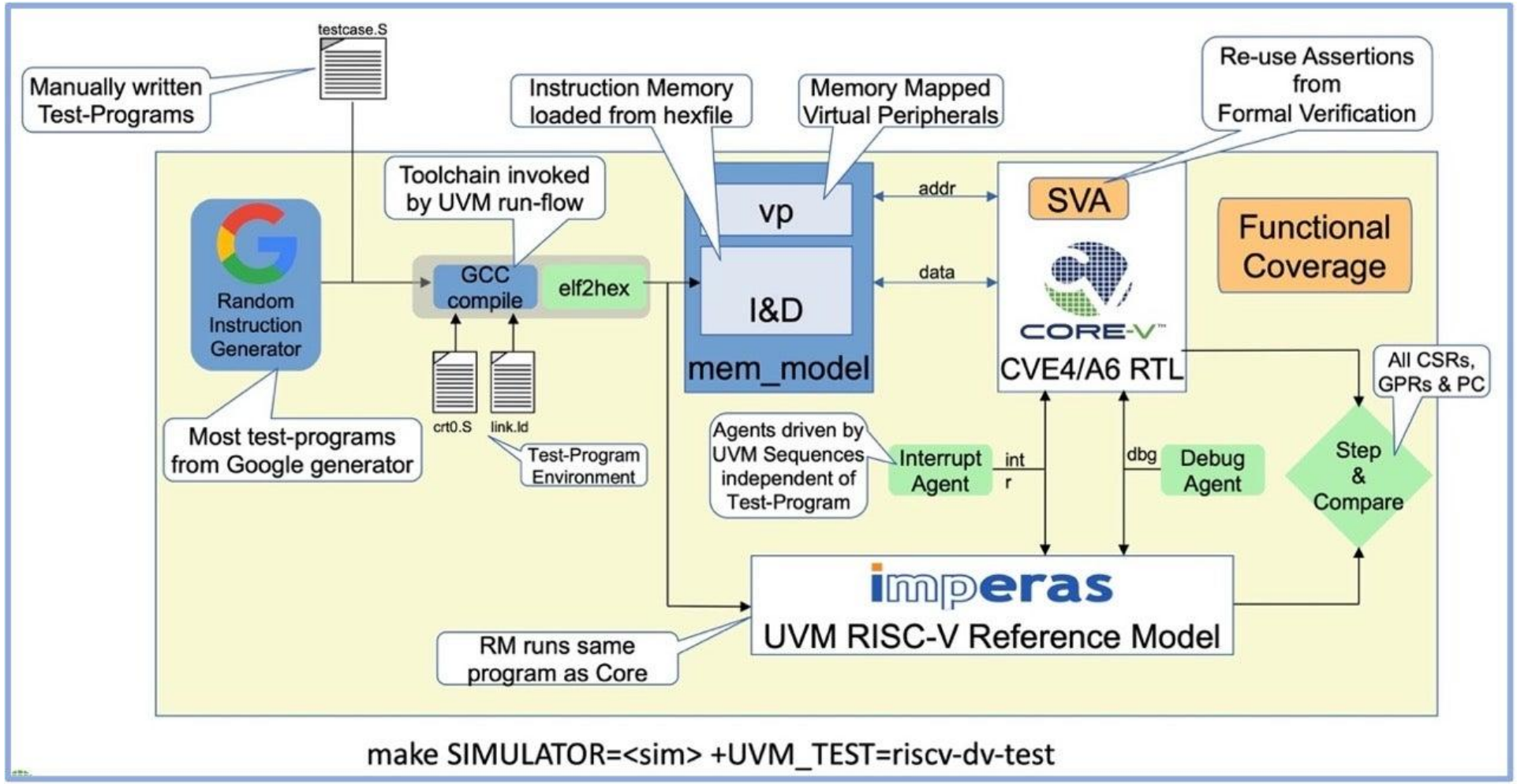


Aside: Trickbox

- A trickbox is a testbench component that lets a test program trigger special events or control its simulation environment.
- Common usecases
  - Generate events (Raise an interrupt or request reset)
  - Control memory responses (Insert delays or inject an access error)
  - Trigger coherency activity (Ask the snoop generator to issue a request)
  - Report test status (Signal pass/fail)
  - Print debug messages

CPU Top-Level Testbench

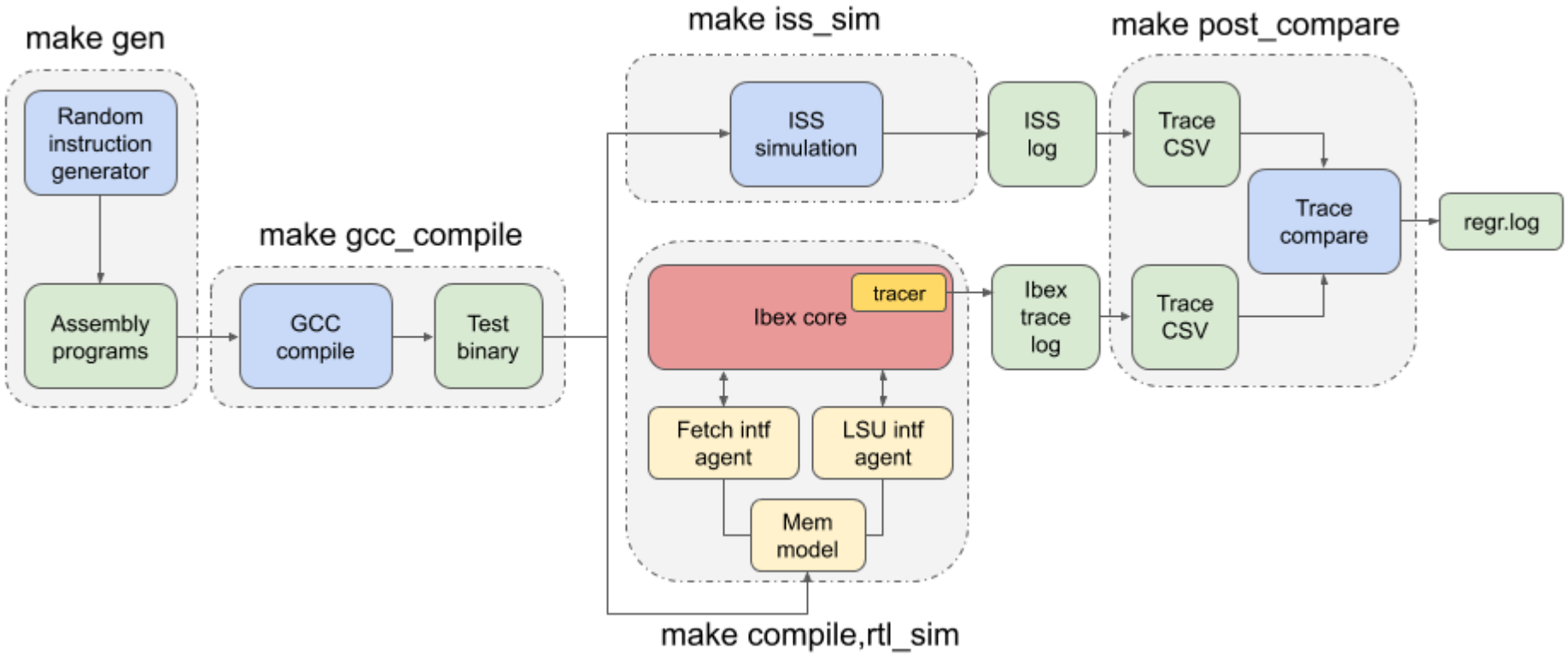
Example from OpenHWGroup



Source: OpenHW Group highlights how verification is a key aspect of the open-source CORE-V processor IP - RISC-V International

CPU Top-Level Testbench

Example from RISC-V Ibex Core Verification



CPU Top-Level: Compliance Suites

- A compliance suite is a collection of tests that checks whether a processor follows its instruction set architecture specification.
  - **Tests:** Instruction behavior, operand corner cases, register updates, and applicable privileged features.
  - **Execution:** Run test programs on the CPU and compare their results against expected architectural behavior.
  - **Common checking method:** Tests write results to a designated memory region (a signature) which is compared with a reference signature.
- Compliance verification is a very small part of from broader CPU verification
- Uses a CPU Top-Level Testbench
  - Simple memory, Ref model integration
  - Tests are binary executable programs

CPU Top Level: Directed Tests

- CPU project teams will create Directed Verification Suite tests to target particular areas of interest
- Unlike compliance suites, these will target microarchitectural features – in particular those crossing unit boundaries (which therefore can't be tested well at unit level)
  - As a result, they will be specific to the processor they are written for
- These are usually hand-written assembler tests; they may be self-checking or rely on an external checking mechanism

CPU Top Level: Random Tests

- ‘Random Instruction Sequence’ tests
- Self-contained assembler tests (usually)
- The tests may be self-checking or rely on an external checking mechanism

Random Instruction Generators

CPU Stimulus is a program

- A CPU consumes instructions, so its stimulus must form meaningful instruction sequences.
- A random instruction generator produces assembly programs that exercise the processor.
- It can randomize:
  - Instructions: arithmetic, loads/stores, branches, CSR accesses.
  - Operands: registers, immediate values, memory addresses.
  - Relationships: dependencies between instructions.
  - Program structure: loops, branches, subroutine calls.

Why not just generate random bits?

- Random bits may decode as illegal instructions and/or produce programs that do little work
- Even individually legal instructions can have issues (e.g. referring to wrong addresses)
- A generator needs constraints for:

Constraint	Example
Supported ISA	Generate only implemented instruction extensions
Register initialization	Initialize address registers before using them
Memory map	Keep ordinary loads/stores within accessible regions
Control flow	Use valid branch targets and bounded loops
Execution environment	Provide setup code, trap handlers, and a termination mechanism

- Illegal instructions and invalid accesses can also be generated deliberately to test exception handling.

Randomize relationships, not just instructions

- Many CPU bugs require interactions between instructions.
  - Register dependencies, instruction orders
  - Needed for verification hazards for example
- Useful generator controls include:
  - Probability of reusing a destination register as a source.
  - Distance between producer and consumer instructions.
  - Frequency of loads, stores, branches, and long-latency operations.
  - Probability of accessing the same memory address.

Directed templates + random details

- Choose an interesting scenario, then randomize many variations of it.

Randomized detail	What it explores
Destination/source registers	Different register combinations
Number of intervening instructions	Different dependency distances
Loaded value and comparison value	Taken and not-taken branches
Address and alignment	Different memory access cases
Memory response delay, driven by the testbench	Different stall durations

- Other useful templates:
  - Store followed by load to the same address.
  - Long-latency operation followed by a dependent instruction.
  - Taken branch with younger instructions that must be flushed.
  - Exception occurring near other in-flight operations.

Example

- Google RISC-V: [chipsalliance/riscv-dv: Random instruction generator for RISC-V processor verification]



Source: chipsalliance/riscv-dv | DeepWiki

Instruction Set Simulators

What is an instruction set simulator?

- An instruction set simulator (ISS) is a software model of a processor's architectural behavior.
- It executes instructions and updates architectural state:
  - Program counter.
  - General-purpose and floating-point registers.
  - Memory.
  - Control/status registers and privilege state.
  - Exceptions and interrupts, when supported.
- An ISS models what instructions do. It typically does not model the pipeline implementation or exact cycle timing.

Difference between RTL CPU and ISS

- The RTL CPU and ISS should produce matching architectural behavior, even though they execute differently.

RTL CPU	ISS reference model
Executes through pipeline stages	Executes using software functions
Handles forwarding, stalls, and flushes	Usually does not model these mechanisms
May execute instructions speculatively	Usually follows architectural execution
Takes implementation-dependent cycles	Usually advances instruction by instruction

How do we use an ISS for checking?

- Run the same program with matching initial state on the RTL CPU and the ISS.
- Compare architectural effects when instructions retire, not pipeline state on every clock cycle.
  - Retirement = when an instruction’s effects become architecturally committed
- Typical comparison information:
  - Retired instruction address and encoding.
  - Destination register and written value.
  - Memory addresses, data, and access effects.
  - Trap information and relevant CSR updates.

How do we use an ISS for checking?

- Two common approaches:

Approach	How it works	Main tradeoff
Trace comparison	Run both independently; compare their execution logs afterward	Simple workflow, but mismatches are discovered after the run
Step-and-compare	Advance the ISS as instructions retire from the RTL; compare their effects during simulation	Detects mismatches immediately, but requires tighter integration

How do we use an ISS for checking?

- The RTL and ISS must agree on the execution environment and the instruction semantics

Challenge	Why it matters
Initial state	Different register or memory contents cause immediate divergence
ISA configuration	Both must implement the same extensions and relevant options
Interrupts and debug requests	The reference model must observe events at the corresponding architectural point
Cycle counters and timers	Values depend on timing that a functional ISS may not reproduce
Memory-mapped I/O	Reads may return changing values or have side effects
Implementation-defined behavior	The model must match the choices allowed and implemented by the DUT

Examples

- Example ISS for RISC-V
  - Whisper
  - Spike
  - Imperas OVPSim

Resources

- openhwfoundation/core-v-verif: Functional verification project for the CORE-V family of RISC-V cores.
- chipsalliance/riscv-dv | DeepWiki
- Verification — Ibex Documentation 0.1.dev50+gc0c070857 documentation